T3000 API & Protocol

Static Resource Copy — Build-Time Asset Pipeline · built Nov 13, 2026

Static Resource Copy — Build-Time Asset Pipeline

Date: 2026-07-10 **Project:** T3000 Webview — EEZ Studio Migration (Electron → Browser) **Topic:** Automatic resource copy from origin repos into Rust build output for release packaging

1. Why

The Rust backend serves three categories of static assets at runtime. None of them live in this repo — they must be copied from sibling repos into the Cargo output tree at build time.

Category	Served at	Content	Origin
Framework sources	`/api/eez-studio/read-text-file?path=...`	C source files for flow runtime compilation	`studio-wasm-libs/release/`
WASM runtimes	`/eez-studio-wasm/wasm/...`	Emscripten-compiled LVGL + dashboard runtimes	`studio-wasm-libs/release/wasm/`
Font assets	`/eez-studio-assets/...`	WOFF/WOFF2/TTF fonts + CSS	`eez-studio/packages/eez-studio-ui/_stylesheets/`

Browser runtime path

The browser has no filesystem access. EEZ Studio code (ported from Electron) uses a browser-stub `fs.promises.readFile()` that resolves to HTTP:

```
TEXT

browser JS → browser-stub fs.promises.readFile()
  → GET /api/eez-studio/read-text-file?path=eez-framework-amalgamation/eez-flow.h
    → Rust server: resolve_path(data_root(), path)
      → T3Web/t3-eez/resources/eez-framework-amalgamation/eez-flow.h
```

`data_root()` (in `api/src/t3_eez_studio/mod.rs`) → `{current_dir}/T3Web/t3-eez/`.

2. Build Script — `api/build.rs`

Runs automatically on every `cargo build` and `cargo build --release`. All destinations resolve to `{profile_root}/T3Web/t3-eez/resources/`.

2.1. Three Copy Helpers

Helper	What it copies	When to use
`copy_resource_dir(manifest_dir, root, relative_src)`	**All files** in a directory → `resources/{dirname}/`	Whole directories (framework sources, docker-build)
`copy_selected_files(manifest_dir, root, relative_src_dir, files, dest_rel)`	**Named files** with optional renaming → `resources/{dest_rel}/`	Picking specific files from a large directory
`copy_wasm_and_fonts(manifest_dir, root)`	All WASM runtimes + font assets (uses `copy_selected_files` internally)	One call for the full WASM + fonts pipeline

2.2. Framework Source Directories (whole-directory copy)

RUST

```rust
// eez-framework-amalgamation — flow runtime C sources
copy_resource_dir(
    &manifest_dir,
    profile_root,
    "../../studio-wasm-libs/release/eez-framework-amalgamation",
);
// → resources/eez-framework-amalgamation/
//     eez-flow.h, eez-flow.cpp, eez-flow-lz4.c, eez-flow-lz4.h, ...

// docker-build — build scripts for LVGL project compilation
copy_resource_dir(
    &manifest_dir,
    profile_root,
    "../../eez-studio/resources/docker-build",
);
// → resources/docker-build/
```

2.3. WASM Runtimes (selected-file copy)

`copy_wasm_and_fonts()` copies three groups:

Dashboard + GUI Lite runtime → `resources/eez-studio-wasm/wasm/`:

```
TEXT

eez_runtime.js, eez_runtime.wasm, eez_gui_lite_runtime.js,
eez_gui_lite_runtime.wasm, lz4.js, lz4.wasm
```

LVGL per-version runtimes → `resources/eez-studio-wasm/wasm/lvgl/{version}/`:

```
TEXT

8.4.0/  lvgl_runtime_v8.4.0.js, lvgl_runtime_v8.4.0.wasm
9.2.2/  lvgl_runtime_v9.2.2.js, lvgl_runtime_v9.2.2.wasm
9.3.0/  lvgl_runtime_v9.3.0.js, lvgl_runtime_v9.3.0.wasm
9.4.0/  lvgl_runtime_v9.4.0.js, lvgl_runtime_v9.4.0.wasm
9.5.0/  lvgl_runtime_v9.5.0.js, lvgl_runtime_v9.5.0.wasm
```

Font assets → `resources/eez-studio-assets/`:

```
TEXT

FontAwesome5-Solid+Brands+Regular.woff, material-icons.css,
MaterialIcons-Regular.woff2, Roboto-Regular.ttf, RobotoMono-Regular.ttf
```

2.4. Complete Output Tree

TEXT

```
target/<profile>/T3Web/t3-eez/resources/
├── eez-framework-amalgamation/      ← copy_resource_dir (studio-wasm-libs)
│   ├── eez-flow.h
│   ├── eez-flow.cpp
│   ├── eez-flow-lz4.c
│   ├── eez-flow-lz4.h
│   ├── eez-flow-sha256.c
│   └── eez-flow-sha256.h
├── docker-build/                    ← copy_resource_dir (eez-studio)
│   ├── Dockerfile
│   └── ...
├── eez-studio-wasm/                 ← copy_wasm_and_fonts (selected files)
│   └── wasm/
│       ├── eez_runtime.js
│       ├── eez_runtime.wasm
│       ├── eez_gui_lite_runtime.js
│       ├── eez_gui_lite_runtime.wasm
│       ├── lz4.js
│       ├── lz4.wasm
│       └── lvgl/
│           ├── 8.4.0/
│           ├── 9.2.2/
│           ├── 9.3.0/
│           ├── 9.4.0/
│           └── 9.5.0/
└── eez-studio-assets/               ← copy_wasm_and_fonts (fonts)
    ├── FontAwesome5-Solid+Brands+Regular.woff
    ├── material-icons.css
    ├── MaterialIcons-Regular.woff2
    ├── Roboto-Regular.ttf
    └── RobotoMono-Regular.ttf
```

---

3. Server Routing

The Rust server (`api/src/server.rs`) mounts each resource directory at a URL prefix:

URL prefix	Serves from	Route function
`/eez-studio-wasm`	`resources/eez-studio-wasm/`	`routes_wasm()`
`/eez-studio-assets`	`resources/eez-studio-assets/`	`routes_assets()`
`/api/eez-studio`	`data_root()` (via `read_text_file` handler)	`bridge_routes()`
`/` (fallback)	SPA dist directory	`routes_static()`

RUST

```rust
// server.rs — create_t3_app()
.nest("/eez-studio-wasm", routes_wasm())
.nest("/eez-studio-assets", routes_assets())
.fallback_service(routes_static())

fn routes_wasm() -> Router {
    Router::new().nest_service("/", get_service(ServeDir::new(
        data_root().join("resources").join("eez-studio-wasm")
    )))
}
```

4. Vite Dev Server Proxy

In development, Vite runs on port 3003 and proxies WASM/asset requests to the Rust backend on port 9103:

JAVASCRIPT

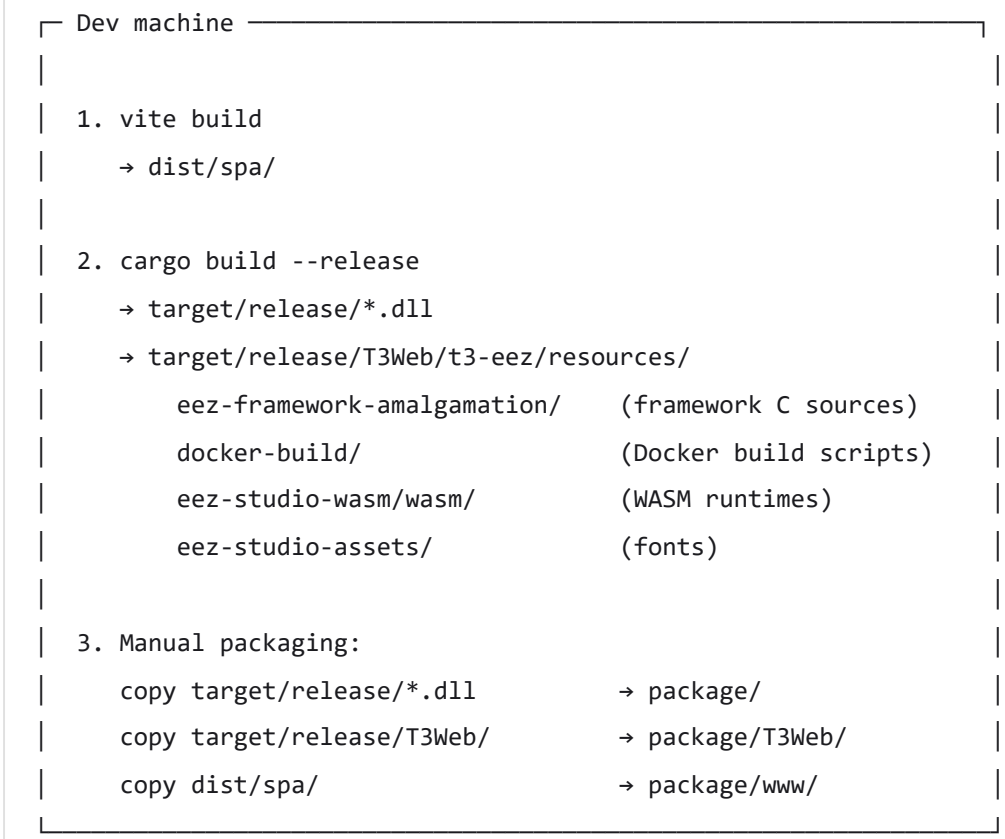
```
// quasar.config.js — extendViteConf
viteConf.server.proxy = {
  "/api/eez-studio": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
  "/eez-studio-wasm": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
  "/eez-studio-assets": {
    target: "http://localhost:9103",
    changeOrigin: true,
  },
};
```

> **Note:** The proxy prefix was renamed from `/t3-eez-studio` to `/eez-studio-wasm` to match the Rust-side naming. Both `build.rs` destination and `server.rs` route use `eez-studio-wasm`.

---

5. Release Packaging Flow

TEXT



Both the DLL and its runtime resources originate from a single `target/release/` tree.

---

6. Relevant Files

File	Role
`api/build.rs`	Three copy helpers; copies all resources into `target/<profile>/T3Web/t3-eez/resources/`
`api/src/server.rs`	Mounts `/eez-studio-wasm`, `/eez-studio-assets`, SPA fallback
`api/src/t3_eez_studio/mod.rs`	`data_root()` → `{cwd}/T3Web/t3-eez/`; `read_text_file` handler
`api/src/t3_eez_studio/bridge_routes.rs`	`/api/eez-studio/*` routes for file read/write
`quasar.config.js`	Vite dev proxy for `/eez-studio-wasm` and `/eez-studio-assets`
`../../studio-wasm-libs/release/`	Origin: framework amalgamation + WASM runtimes
`../../eez-studio/resources/docker-build/`	Origin: Docker build scripts
`../../eez-studio/packages/eez-studio-ui/_stylesheets/`	Origin: font assets

7. Adding More Static Resources

Whole directory (all files):

RUST

```rust
// In build.rs → main()
copy_resource_dir(
    &manifest_dir,
    profile_root,
    "../../some-repo/path/to/dir",
);
// → resources/dir/
```

Specific files (with optional renaming):

```rust
copy_selected_files(
    &manifest_dir,
    profile_root,
    "../../some-repo/path/to/source",
    &[
        ("source_name.ext", "sub/dest_name.ext"),
    ],
    "my-dest-dir",  // → resources/my-dest-dir/
);
```

New server route:

```rust
// In server.rs → create_t3_app()
.nest("/my-prefix", routes_my_prefix())

fn routes_my_prefix() -> Router {
    Router::new().nest_service("/", get_service(ServeDir::new(
        data_root().join("resources").join("my-dest-dir")
    )))
}
```

New Vite proxy:

```javascript
// In quasar.config.js → extendViteConf → viteConf.server.proxy
"/my-prefix": {
    target: "http://localhost:9103",
    changeOrigin: true,
},
```